

ARTI403 — Image Processing

Lab 1 — Basics of Programming with Python

Session 1 · Basics of programming with Python Digital image representation and manipulation in Python — loading, displaying, saving, and inspecting images using OpenCV, PIL, Matplotlib, and NumPy.

Duration 100 minutes

Outcome Explain how digital images are represented and manipulated in a computer

Tools Python 3, Anaconda (Jupyter / Spyder)

How to work through the notebook

- The lab is organised into the **Tasks** from the procedural manual (Task #1 – Task #4), followed by the **Assessment**.
 - Run the cells **in order**, top to bottom — Tasks #3 and #4 reuse the `img` and `img2` variables created in Task #2.
 - Task #2 onward needs two image files that are **not included** in this notebook — see the note right before Task #2 for exactly where to place them.
 - The **Assessment** is at the end of the notebook.
-

By the end of this lab you should be able to

1. Load an image using **OpenCV** (`cv2.imread`) and using **PIL** (`Image.open`).
 2. Display an image with **Matplotlib**.
 3. Save an image back to disk using both OpenCV and PIL.
 4. Represent an image as a **NumPy array** and inspect its shape and values.
-

Setup

Run this cell first. It imports every library used in this lab and confirms they are installed (Steps #3–#4 of the manual: OpenCV, Pillow, Matplotlib and NumPy).

```
In [13]: # If a library below is missing, install it first (uncomment as needed):  
# pip install opencv-python pillow matplotlib numpy  
  
import sys  
import cv2  
import numpy as np  
import PIL
```

```
from PIL import Image
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.cm as cm

print("Python      :", sys.version.split()[0])
print("OpenCV      :", cv2.__version__)
print("Pillow       :", PIL.__version__)
print("Matplotlib    :", matplotlib.__version__)
print("NumPy         :", np.__version__)

print("\nSetup OK")
```

```
Python      : 3.14.3
OpenCV      : 5.0.0
Pillow      : 12.3.0
Matplotlib  : 3.11.1
NumPy       : 2.5.2
```

Setup OK

Task #1 — Exercise with Spyder

This task is self-practice, not a coding task inside this notebook: work through the **two cheat sheets** listed in the manual's *Resources* section (**Python Basics** and **NumPy**) inside Spyder to get comfortable with the syntax used in the rest of this lab.

Task #2 — Loading and Visualizing Images

Before running the cells below: create a folder named `images` next to this notebook and place the two files below inside it. These are standard grayscale test images used throughout image-processing courses — ask your instructor for them if you do not already have them from the course resources.

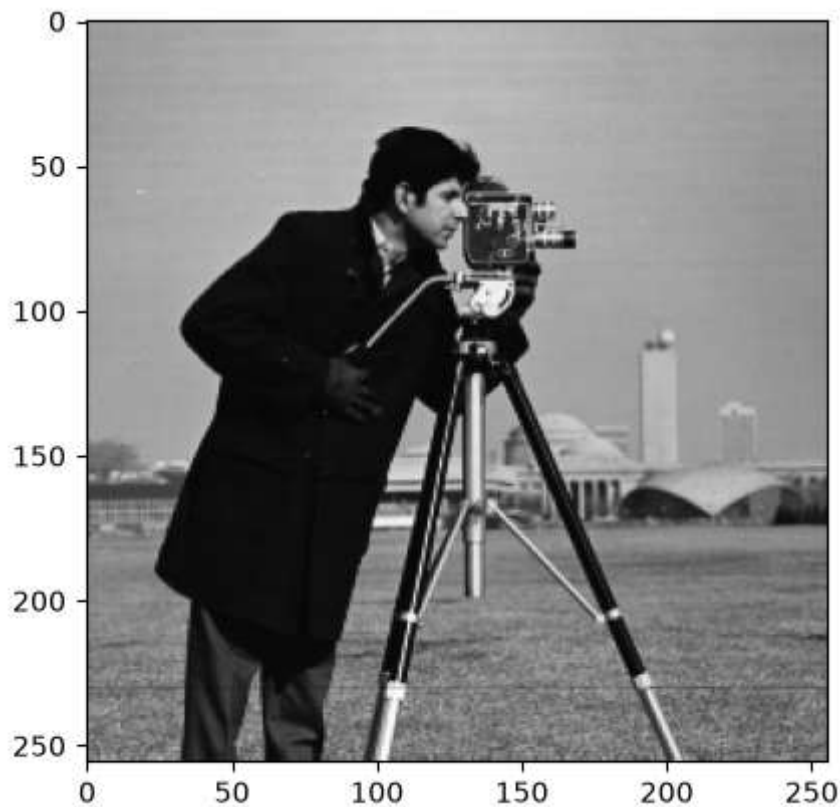
- `images/cameraman.tif`
- `images/lena_gray_256.tif`

a. Loading/reading an image using OpenCV

`cv2.imread` loads the image file into a NumPy array, which is then displayed with `plt.imshow`.

```
In [14]: img = cv2.imread('images/cameraman.tif') # Load the image data into a NumPy array

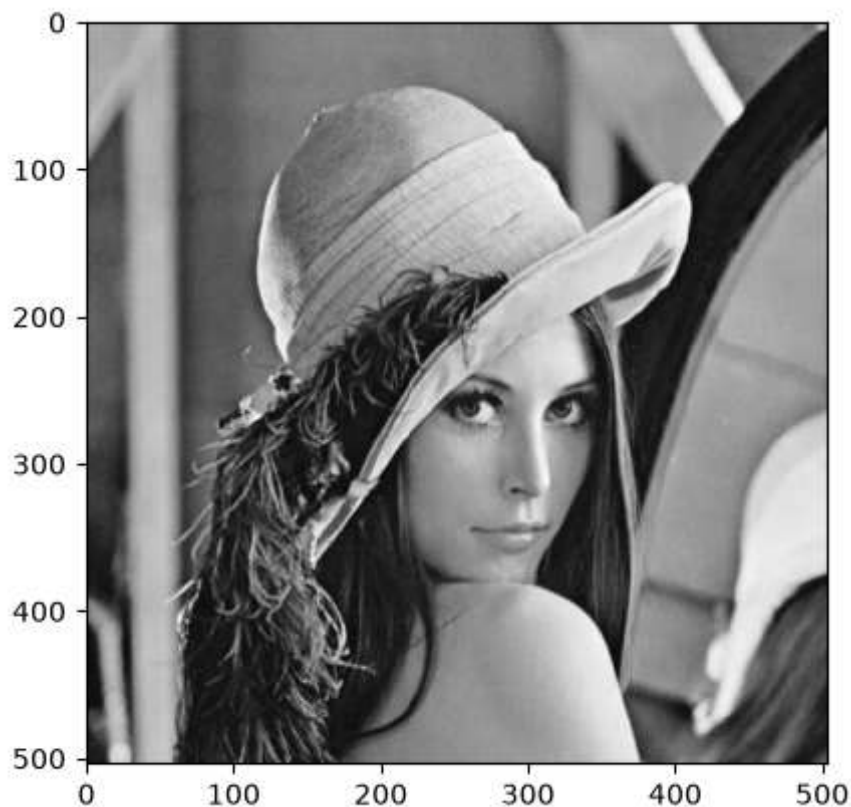
plt.imshow(img)
plt.show()
```



b. Loading/reading image using PIL

`Image.open` loads the file into a PIL `Image` object. Since the image is grayscale, it is displayed with the `Greys_r` colormap so Matplotlib does not tint it.

```
In [15]: img2 = Image.open('images/lena_gray_256.tif') # Load image into a PIL Image obj
plt.imshow(img2, cmap=cm.Greys_r)
plt.show()
```



Task #3 — Image Storing

`img` (loaded with OpenCV) and `img2` (loaded with PIL) from Task #2 can each be written back to disk.

a. Saving an image to disk using OpenCV

`cv2.imwrite` writes a NumPy image array to disk, choosing the format from the file extension.

```
In [16]: cv2.imwrite('new_image.jpg', img)
```

```
Out[16]: True
```

b. Saving an image to disk using PIL

PIL images have their own `save` method, which also chooses the format from the file extension.

```
In [17]: img2.save('new_image2.jpg')
```

Task #4 — Display Image as an Array

An image loaded with OpenCV is already a NumPy array. A PIL image can be converted into one with `np.array`.

a. The OpenCV image as an array

`img.shape` gives its dimensions, and printing `img` shows the raw pixel values.

```
In [18]: print(img.shape) # Note: img loaded using OpenCV is a NumPy array  
print(img)
```

```

(256, 256, 3)
[[[156 156 156]
  [159 159 159]
  [158 158 158]
  ...
  [151 151 151]
  [152 152 152]
  [152 152 152]]

[[[160 160 160]
  [154 154 154]
  [157 157 157]
  ...
  [154 154 154]
  [155 155 155]
  [153 153 153]]

[[[156 156 156]
  [159 159 159]
  [158 158 158]
  ...
  [151 151 151]
  [152 152 152]
  [152 152 152]]

...

[[[114 114 114]
  [132 132 132]
  [123 123 123]
  ...
  [135 135 135]
  [137 137 137]
  [114 114 114]]

[[[121 121 121]
  [126 126 126]
  [130 130 130]
  ...
  [133 133 133]
  [130 130 130]
  [113 113 113]]

[[[121 121 121]
  [126 126 126]
  [130 130 130]
  ...
  [133 133 133]
  [130 130 130]
  [113 113 113]]]]

```

b. Converting the PIL image to a NumPy array

`np.array()` converts the PIL image into a NumPy array so it can be inspected the same way.

```
In [19]: # Convert the image to a NumPy array
img_array = np.array(img2)
```

```
# Print the array
print(img_array.shape)
print(img_array)
```

```
(504, 504, 3)
[[[159 175 174]
  [159 175 174]
  [159 175 174]
  ...
  [170 189 185]
  [146 164 160]
  [114 133 129]]

 [[159 175 174]
  [159 175 174]
  [159 175 174]
  ...
  [170 189 185]
  [146 164 160]
  [114 133 129]]

 [[159 175 174]
  [159 175 174]
  [159 175 174]
  ...
  [172 188 185]
  [146 164 160]
  [114 133 129]]

 ...

 [[ 24  24  24]
  [ 22  22  22]
  [ 26  29  28]
  ...
  [ 85  96  97]
  [ 78  91  91]
  [ 82  96  96]]

 [[ 21  21  21]
  [ 21  21  21]
  [ 28  30  29]
  ...
  [ 87 101 101]
  [ 84  97  97]
  [ 91 104 104]]

 [[ 19  19  19]
  [ 20  20  20]
  [ 29  31  30]
  ...
  [ 86 100 100]
  [ 86 100 100]
  [ 96 110 110]]]
```

Assessment

The task

As stated in the manual, you will be asked to perform different operations on the array (e.g. `img_array` from Task #4) using the NumPy library. The manual does not list the specific operations — they will be given separately, for example by your instructor in class.

```
In [20]: # The specific NumPy array operations for the Assessment are not listed in the m  
# Perform the operation(s) given by your instructor here, using img_array (or im
```

End of notebook.